Gauntlet: Aera Contracts v3

Security Review

Cantina Managed review by:
Devtooligan, Lead Security Researcher
Jonatas Martins, Security Researcher

April 15, 2026

Contents

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Gauntlet's applied research and optimization teams model market risk, economic incentives, and drive sustainable growth for crypto's top protocols, chains, and onchain treasuries.

From Mar 17th to Mar 26th the Cantina team conducted a review of aera-contracts-v3 on commit hash fda89451. The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	3	3	0
Low Risk	3	2	1
Gas Optimizations	0	0	0
Informational	3	3	0
Total	**9**	**8**	**1**

2.1 Scope

The security review had the following components in scope for aera-contracts-v3 on commit hash fda89451:

```
src
├── core
│   ├── Constants.sol
│   ├── PriceAndFeeCalculator.sol
│   ├── Provisioner.sol
│   ├── Types.sol
│   └── interfaces
│       ├── IPriceAndFeeCalculator.sol
│       └── IProvisioner.sol
└── periphery
    └── AeraV3MorphoV2Adapter.sol
```

3 Findings

3.1 Medium Risk

3.1.1 `withdraw()` and `redeem()` use inconsistent numeraire conversion for epoch cap accounting

Severity: Medium Risk

Context: Provisioner.sol#L641

Summary: `withdraw()` and `redeem()` compute `redeemNumeraire` differently for epoch cap accounting. `redeem()` uses `convertUnitsToNumeraire(unitsIn)` (the full unit-side value before multiplier discount), while `withdraw()` uses `convertTokenToNumeraire(tokensOut)` (the discounted token-side value). This inconsistency means equivalent redemptions consume different amounts of epoch cap depending on which function is called.

Description: In `redeem()` at `src/core/Provisioner.sol:609`, the redeem value is computed as:

```
uint256 redeemNumeraire =
↪   PRICE_FEE_CALCULATOR.convertUnitsToNumeraire(MULTI_DEPOSITOR_VAULT, unitsIn);
```

In `withdraw()` at `src/core/Provisioner.sol:641`, the redeem value is computed as:

```
uint256 redeemNumeraire =
↪   PRICE_FEE_CALCULATOR.convertTokenToNumeraire(MULTI_DEPOSITOR_VAULT, token,
↪   tokensOut);
```

Both values feed into `_prepareSyncRedeem()` which uses `redeemNumeraire` for two purposes: dynamic premium calculation and epoch cap accounting. Since `tokensOut` reflects the multiplier discount (e.g., at `syncRedeemMultiplier = 9500`, 1000 units produces 950 tokens), `withdraw()` charges approximately 5% less against the epoch cap for an equivalent redemption.

Additionally, the smaller `redeemNumeraire` from `withdraw()` feeds into `_computeDynamicPremiumBps()`, understating the utilization ratio and suppressing the dynamic premium charged on the current and subsequent redeems in the epoch.

Impact Explanation: The impact is low. The two functions charge different amounts against the epoch cap for equivalent redemptions. `redeem()` overcharges relative to actual tokens leaving the vault (since `unitsIn` is pre-discount), while `withdraw()` charges based on the actual outflow. The magnitude of the discrepancy scales with the multiplier discount - at 9500 BPS, the difference is approximately 5%.

Likelihood Explanation: The likelihood is high. Both `withdraw()` and `redeem()` are public functions available to any depositor. The asymmetry applies automatically whenever `syncRedeemMultiplier` is below 10000 or a dynamic premium is active, which is the normal operating configuration for sync redeems.

Recommendation: Consider splitting the numeraire values in `withdraw()` so that `redeemNumeraire` (token-based) is used for dynamic premium calculation and a separate `epochRedeemedNumeraire` (unit-based, matching `redeem()`) is used for epoch cap accounting:

```
  unitsIn = _tokensToUnitsCeilIfActive(token, tokensOut, effectiveMultiplier);
  // Requirements: units in does not exceed max units in
  require(unitsIn <= maxUnitsIn, Aera__MaxUnitsInExceeded());

+ // Interactions: recompute numeraire from units for consistent epoch cap accounting
↪   with redeem()
+ uint256 epochRedeemedNumeraire =
↪   PRICE_FEE_CALCULATOR.convertUnitsToNumeraire(MULTI_DEPOSITOR_VAULT, unitsIn);
+
  // Requirements + Effects + Interactions: sync redeem
- _syncRedeem(token, tokensOut, unitsIn, receiver, redeemNumeraire, epochCapNumeraire,
↪   epochRedeemedNumeraire);
+ _syncRedeem(token, tokensOut, unitsIn, receiver, epochRedeemedNumeraire,
↪   epochCapNumeraire, epochRedeemedNumeraire);
```

This adds an extra `PriceAndFeeCalculator` call but makes epoch cap accounting consistent across both paths. The `_syncRedeem()` signature would need to accept the separate value for epoch cap tracking.

Gauntlet: Fixed in PR 580.

Cantina Managed: PR 580 resolves the inconsistency by standardizing epoch cap accounting on the `withdraw()` approach, using `tokensOut`-based numeraire (the actual token outflow) as the consistent measure rather than the pre-discount `unitsIn`-based value used by `redeem()`.

3.1.2 Zero cost share mints are possible with non-numeraire tokens pricing

Severity: Medium Risk

Context: Provisioner.sol#L221, Provisioner.sol#L648

Description: The `mint()` and `_solveDepositVaultFixedPrice()` functions rely on `_unitsToTokensCeilIfActive()` in `src/core/Provisioner.sol`, which calls `convertUnitsToTokenIfActive(..., Math.Rounding.Ceil)`. In `src/core/PriceAndFeeCalculator.sol`, however, the `Ceil` rounding only applies to the `units` → `numeraire` conversion. The final `numeraire` → `token` conversion is delegated through `OracleRegistry.getQuoteForUser()`, and the oracle interface requires that quote to round down.

Because of that, a positive `unitsOut` can still produce a `0` token quote for sufficiently small share mints in high-value non-numeraire assets. `mint()` only requires `unitsOut != 0` and `maxTokensIn != 0`, then checks `tokensIn <= maxTokensIn`. If the computed `tokensIn` is `0`, the call still succeeds and `_syncDeposit()` routes into `MultiDepositorVault.enter()`, which skips the token transfer when `tokenAmount == 0` but still mints the requested units.

Example:

- Numeraire = USDC, `6` decimals.
- Deposit token = WBTC, `8` decimals.
- `1 WBTC = 100,000 USDC`.
- `1 share = 1 USDC`.
- User calls `mint(WBTC, 1e12, 1, alice)`.

`1e12` share units equals `0.000001` share, so the `units` → `numeraire` rounds up to `1` USDC base unit. At that price, one satoshi is worth `1000` USDC base units, so converting `1` USDC base unit into WBTC rounds down to `0`. `tokensIn` therefore becomes `0`, `mint()` still passes, and `enter()` mints `1e12` share units to `alice` without transferring WBTC into the vault.

The same rounding asymmetry also appears on the `withdraw()` path. `withdraw()` relies on `_tokensToUnitsCeilIfActive()` in `src/core/Provisioner.sol`, but `convertTokenToUnitsIfActive(..., Math.Rounding.Ceil)` still floors the non-numeraire `token` → `numeraire` leg through the oracle in `src/core/PriceAndFeeCalculator.sol`. That means very small non-numeraire withdrawals can compute `unitsIn = 0`, after which `MultiDepositorVault.exit()` can burn zero units and still transfer `tokensOut`. So this is not only a deposit-side issue; it affects exact-output pricing in both directions.

Recommendation: Reject zero-amount pricing results, or add roundings to oracle quotes for deposits and withdrawals.

Gauntlet: Fixed in PR 580.

Cantina Managed: Fix verified.

3.1.3 Inconsistent dynamic premium basis between `redeem()` and `withdraw()`

Severity: Medium Risk

Context: Provisioner.sol#L609-L613, Provisioner.sol#L641-L645

Description: `redeem()` and `withdraw()` compute the sync redeem dynamic premium from different economic bases, which can cause equivalent exits to receive different pricing. In `redeem()`, the premium input passed into `_prepareSyncRedeem()` is derived from the unit-side value:

```
convertUnitsToNumeraire(..., unitsIn)
```

In `withdraw()`, the premium input passed into `_prepareSyncRedeem()` is derived from the token-side value:

```
convertTokenToNumeraire(..., tokensOut)
```

Because `_computeDynamicPremiumBps()` uses this value to measure utilization, the two paths can compute different dynamic premiums for what should be the same exit. When premium is active, the realized token outflow is smaller than the gross unit-side value, so `withdraw()` can treat the trade as smaller than the equivalent `redeem()`. As a result, `withdraw()` may receive a lower premium and require fewer units than the economically equivalent `redeem()` path.

Simple example:

- Assume all assets are priced `1:1` for simplicity.
- Assume premium is active and utilization dominates pricing.

Then:

- `redeem(100 units)` computes premium from `100` numeraire.
- After premium, the user receives `80` tokens.

From the exact same starting state:

- `withdraw(80 tokens)` computes premium from `80` numeraire.
- Because `80 < 100`, the path sees lower utilization and applies a smaller premium.
- The result may be that `withdraw(80 tokens)` burns only `95 units` instead of `100`.

These two operations should be near-inverses of each other, but under the current implementation they can diverge materially because the premium is computed from different trade bases.

Proof of Concept: Using the `Provisioner.t.sol` test:

```
function test_redeem_withdraw_dynamicPremium_mismatch() public {
    uint16 maxDynamicPremiumBps = 4999;
    uint16 syncRedeemMultiplier = uint16(ONE_IN_BPS);
    uint256 unitsIn = 4998e18;
    uint256 priceAge = 10;

    vm.prank(users.owner);
    provisioner.setSyncRedeemDetails(
        SYNC_REDEEM_MAX_PRICE_AGE, SYNC_REDEEM_RELATIVE_CAP_BPS,
        ↪  SYNC_REDEEM_ABSOLUTE_CAP, maxDynamicPremiumBps
    );

    vm.prank(users.owner);
    provisioner.setTokenDetails(
        token,
        TokenDetails({
            asyncDepositEnabled: true,
            asyncRedeemEnabled: true,
            syncDepositEnabled: true,
            syncRedeemEnabled: true,
            asyncDepositMultiplier: uint16(ONE_IN_BPS),
            asyncRedeemMultiplier: uint16(ONE_IN_BPS),
            syncDepositMultiplier: uint16(ONE_IN_BPS),
            syncRedeemMultiplier: syncRedeemMultiplier,
            pushFundsSubmitDataPointer: address(0),
            pullFundsSubmitDataPointer: address(0)
        })
    );

    _mockGetAnchorTimestamp(uint32(block.timestamp - priceAge));
    _mockGetVaultValueAtLastUpdate(uint256(SYNC_REDEEM_LAST_TOTAL_SUPPLY));
    _mockConvertUnitsToToken(token, unitsIn, unitsIn);
    _mockConvertUnitsToNumeraire(unitsIn, unitsIn);
```

```
    //This are already calculated expected outputs through the setup for easily 1:1
    ↪  convertions
    uint256 expectedTokensOut = 3_748_999_800_000_000_000_000;
    uint256 expectedWithdrawUnitsIn = 4_614_153_600_000_000_000_000;

    // Mock same-state withdraw inputs derived from the expected redeem output.
    //This is necessary for conversion 1:1
    _mockConvertTokenToNumeraire(expectedTokensOut, expectedTokensOut);
    _mockConvertTokenToUnits(token, expectedWithdrawUnitsIn, expectedWithdrawUnitsIn);

    multiDepositorVault.mint(users.alice, ALICE_VAULT_UNITS);
    ERC20Mock(address(token)).mint(address(multiDepositorVault), 100_000e18);

    uint256 snapshotId = vm.snapshotState();

    vm.prank(users.alice);
    uint256 redeemTokensOut = provisioner.redeem(token, unitsIn, 1, users.bob);

    assertTrue(vm.revertToState(snapshotId));

    vm.prank(users.alice);
    uint256 withdrawUnitsIn = provisioner.withdraw(token, redeemTokensOut,
    ↪  type(uint256).max, users.bob);

    assertEq(unitsIn - withdrawUnitsIn, 383_846_400_000_000_000_000);
}
```

Recommendation: There is no straight forward fix for this issue. We need to use the same premium basis for both paths. The cleanest approach is to make both `redeem()` and `withdraw()` compute dynamic premium from the token-side realized outflow, since `withdraw()` already uses that basis and `redeem()` epoch accounting is already token-side. This removes the pricing asymmetry between the two entrypoints.

Gauntlet: Fixed in commit 13f71c32.

Cantina Managed: Fix verified. The commit removes the mismatched premium basis, so `redeem()` and `withdraw()` now use the same premium logic.

3.2 Low Risk

3.2.1 `refundDeposit()` does not pull funds from yield

Severity: Low Risk

Context: Provisioner.sol#L251

Summary: When `pushFundsSubmitDataPointer` is configured, `deposit()` automatically pushes deposited tokens from the vault to a yield source in the same transaction. A subsequent call to `refundDeposit()` will revert because the vault lacks sufficient idle token balance to fulfill the refund, and `refundDeposit()` does not call `_pullFundsIfNeeded()` beforehand.

Description: The `deposit()` function at `src/core/Provisioner.sol:171-199` calls `_syncDeposit()` to transfer tokens from the depositor into the vault, then calls `_pushFundsIfConfigured()` which moves those tokens from the vault to an external yield source (e.g., Aave) via `vault.submit()`. After this, the vault's idle token balance is near zero.

When the owner later calls `refundDeposit()` to return tokens to the depositor, the function calls `vault.exit()` at line 251, which attempts `token.safeTransfer(receiver, tokenAmount)` from the vault. This transfer reverts because the tokens are no longer in the vault.

The correct pattern exists in `_syncRedeem()` at line 1052, which calls `_pullFundsIfNeeded()` before `vault.exit()`. This helper checks the vault's idle balance and, if insufficient, reads the pull-funds submit data from SSTORE2 and calls `vault.submit()` to retrieve tokens from the yield source. `refundDeposit()` is the only token-transferring `vault.exit()` call path that lacks this pull mechanism.

Recommendation: Consider adding a `_pullFundsIfNeeded()` call before the `vault.exit()` in `refundDeposit()`, matching the pattern used in `_syncRedeem()`.

Gauntlet: Fixed in PR 580.

Cantina Managed: Fix verified.

3.2.2 Receiver can bypass refund timeout by redeeming locked units

Severity: Low Risk

Context: Provisioner.sol#L603

Summary: A receiver with locked vault units can bypass the refund timeout mechanism by calling `redeem()` or `withdraw()` instead of waiting for `refundDeposit()`. Burns are not blocked by the lock check in `_update()`, allowing the receiver to exit with their units (minus fees) while the depositor loses the ability to claw back the deposit.

Description: When a sync deposit is made with a receiver different from the sender, the Provisioner sets `userUnitsRefundableUntil[receiver]` at `src/core/Provisioner.sol:989` to lock the receiver's units during the refund window. This lock is intended to give the depositor (or owner) time to call `refundDeposit()` to reverse the deposit if needed.

The lock check in `MultiDepositorVault._update()` at `src/core/MultiDepositorVault.sol:124-125` exempts burns (`to == address(0)`). Since `redeem()` and `withdraw()` both call `vault.exit()` which burns units, the receiver can redeem their locked units at any time during the refund window. The receiver pays the applicable sync redeem fees (multiplier discount and dynamic premium), but the depositor's refund right is effectively defeated.

Additionally, the lock mechanism is account-level rather than per-deposit. An account with any locked units has ALL units blocked from non-burn transfers, including units that predate the lock. The client acknowledged this as a deliberate design simplification.

Impact Explanation: A receiver who has been granted units via a third-party sync deposit can extract the value before the depositor has a chance to call `refundDeposit()`. The receiver pays sync redeem fees, which provides some economic friction, but the refund mechanism does not function as intended. The depositor's `refundDeposit()` call would then fail since the units have already been burned.

Likelihood Explanation: This requires a sync deposit with `sender != receiver` and the receiver to act within the refund window. The receiver must have an approved deposit or the sender must have explicitly designated them, making this a targeted scenario rather than an opportunistic attack.

Recommendation: Consider adding a lock check in `redeem()` and `withdraw()` (or in `_syncRedeem()`) that prevents users with locked units from burning during the refund window. This would make the lock check consistent across burns and transfers:

```
  function _syncRedeem(
      IERC20 token,
      uint256 tokensOut,
      uint256 unitsIn,
      address receiver,
      uint256 redeemNumeraire,
      uint256 epochCapNumeraire,
      uint256 epochRedeemedNumeraire
  ) internal {
+     // Requirements: caller's units must not be locked (refund window)
+     require(userUnitsRefundableUntil[msg.sender] < block.timestamp,
 ↪  Aera__UnitsLocked());
+
      // Requirements + Effects: check epoch cap and track redemption
      _requireEpochCapNotExceeded(redeemNumeraire, epochCapNumeraire,
       ↪  epochRedeemedNumeraire);
```

Gauntlet: Fixed in PR 580.

Cantina Managed: Fix verified. The `redeem()` and `withdraw()` functions check for the `userUnitsRefundableUntil[msg.sender] < block.timestamp`.

3.2.3 Refundable sync deposits can become fee-bearing before refund

Severity: Low Risk

Context: Provisioner.sol#L233

Description: A sync deposit transfers tokens into the vault immediately through `_syncDeposit()` in `src/core/Provisioner.sol`, but it remains refundable for `depositRefundTimeout` through `refundDeposit()`. If the accountant posts a new anchor price before that refund window closes, `PriceAndFeeCalculator.setAnchorPrice()` can accrue fees against vault state that still includes the refundable deposit. Those fees can then be claimed from the vault's real `FEE_TOKEN` balance through `FeeVault.claimFees()`, while `refundDeposit()` still expects to return the original `tokenAmount` in full.

Example:

- The vault has `1,000,000` shares and `1,000,000 USDC`.
- Anchor price and high-water mark are both `1.00 USDC/share`.
- Sync deposit multiplier is `9990`.
- A user makes a refundable sync deposit of `100,000 USDC`.

The vault receives the full `100,000 USDC`, but the user receives only `99,900` shares, so the implied unit price increases to about `1.0000909 USDC/share`. If the accountant posts that higher price before the refund window ends, `_accrueFees()` can record performance fees on the pre-existing supply and those fees can be claimed in USDC. If the deposit is later refunded, the protocol still tries to return the original `100,000 USDC`, but the fees taken during the refund window are not unwound. In practice, that means the refund must be funded from the remaining vault balance, which can dilute other holders or revert if the vault no longer has enough of the refund asset.

Recommendation: Do not allow refundable sync deposits to contribute to finalized fee claims before their refund window has expired.

Gauntlet: Acknowledged. Refunds are intended to be extremely rare and targeted at arbitrageurs so we will not fix this.

Cantina Managed: Acknowledged.

3.3 Informational

3.3.1 Redeem cancellations can be enabled with zero cap

Severity: Informational

Context: Provisioner.sol#L466

Summary: The validation in `setCancellationDetails()` allows enabling redeem cancellations with `redeemCancellationCapNumeraire == 0` when the dynamic fee cap is also zero. This creates a state where redeem self-cancellation is nominally enabled but every attempt reverts, since `requestNumeraire` is always positive and fails the `requestNumeraire <= 0` cap check.

Description: In `setCancellationDetails()` at `src/core/Provisioner.sol:463-468`, the validation when `redeemCancellationsEnabled == true` only requires `redeemCancellationCapNumeraire != 0` when `redeemCancellationDynamicFeeCapNumeraire != 0`. This means the owner can enable redeem cancellations with both values set to zero.

When a user attempts to self-cancel a redeem request via `cancelRequest()`, the function checks at line 356:

```
require(requestNumeraire <= _redeemCancellationCapNumeraire,
↪  Aera__RequestAmountExceedsRefundCap());
```

Since `requestNumeraire` is derived from `convertUnitsToNumeraire()` and is always positive for any valid redeem request, this check always fails when `_redeemCancellationCapNumeraire == 0`.

Recommendation: Consider simplifying the validation to unconditionally require a non-zero cap when redeem cancellations are enabled:

```
  if (redeemCancellationsEnabled) {
-     // Requirements: enabled dynamic redeem cancellation fee requires a non-zero
↪  cancellation cap denominator
-     require(
-         redeemCancellationDynamicFeeCapNumeraire == 0 || redeemCancellationCapNumeraire
↪  != 0,
-         Aera__RedeemCancellationCapNumeraireZero()
-     );
+     // Requirements: enabled redeem cancellations require a non-zero cancellation cap
+     require(redeemCancellationCapNumeraire != 0,
↪  Aera__RedeemCancellationCapNumeraireZero());
  }
```

Gauntlet: Fixed in PR 580.

Cantina Managed: Fix verified.

3.3.2 Morpho adapter sync deposits lock the adapter and block async withdraw requests

Severity: Informational

Context: Provisioner.sol#L989

Description: When the Morpho adapter performs a sync deposit via `Provisioner.deposit()` with `address(this)` as the receiver, the Provisioner unconditionally overwrites `userUnitsRefundableUntil[adapter]` at `src/core/Provisioner.sol:989`. This sets an account-level lock on the adapter's vault units for the configured `depositRefundTimeout` duration.

While locked, `requestWithdraw()` on the adapter calls `Provisioner.requestRedeem()`, which performs a `safeTransferFrom()` to move vault units from the adapter to the Provisioner. This transfer triggers the lock check in `MultiDepositorVault._update()` at `src/core/MultiDepositorVault.sol:125` and reverts with `Aera__UnitsLocked()`. Repeated sync deposits extend the lock indefinitely since the timestamp is overwritten on each deposit.

Sync redeems via `redeem()` and `withdraw()` are unaffected because burns bypass the lock check (`to == address(0)` at line 124).

The project team confirmed this is not an issue when the allocator is the only actor and there are changes in flight for the liquid adapter use case.

Recommendation: Consider documenting the operational constraint clearly: when the adapter is used as a liquid adapter with sync redeems enabled, `syncDepositAllowed` must remain `false` to prevent external users from locking the adapter.

Gauntlet: Fixed in PR 580.

Cantina Managed: Fix verified.

3.3.3 `solveRequestsVault()` does not check PFC vault pause

Severity: Informational

Context: Provisioner.sol#L391

Summary: `solveRequestsVault()` does not check `isVaultPaused()` before executing `preSolveSubmitData` and `postSolveSubmitData`. A solver can call this function with an empty request array and still execute arbitrary vault submit calls while the vault is paused in the `PriceAndFeeCalculator`. The companion function `solveRequestsDirect()` has an explicit pause check, creating an inconsistency.

Description: `solveRequestsVault()` at `src/core/Provisioner.sol:384-403` accepts `preSolveSubmitData` and `postSolveSubmitData` parameters that are forwarded to `vault.submit()`. These submit calls execute regardless of whether the vault is paused in the PFC. The function does check `solvingNotPaused(token)` via the solving gate, but this is a separate pause mechanism from the PFC vault pause.

By contrast, `solveRequestsDirect()` at line 408-410 explicitly checks `PRICE_FEE_CALCULATOR.isVaultPaused()` before processing any requests. This means the PFC vault pause, which is intended to halt vault operations when prices are suspect, can be bypassed by a solver calling `solveRequestsVault()` with an empty `requests` array while still executing submit calls.

The solver role is authorized (`requiresAuth`) and the submit calls are Merkle-constrained by the vault's guardian root, which limits what operations can be performed. The client confirmed the solver is not intended to act as a guardian and agreed the check should be added.

Recommendation: Consider adding a PFC vault pause check at the start of `solveRequestsVault()`, consistent with `solveRequestsDirect()`:

```
  function solveRequestsVault(
      IERC20 token,
      Request[] calldata requests,
      bytes calldata preSolveSubmitData,
      bytes calldata postSolveSubmitData
  ) external requiresAuth nonReentrant solvingNotPaused(token) {
+     // Requirements: vault is not paused in the priceAndFeeCalculator
+     require(!PRICE_FEE_CALCULATOR.isVaultPaused(MULTI_DEPOSITOR_VAULT),
 ↪  Aera__PriceAndFeeCalculatorVaultPaused());
+
      // Interactions: pre-solve submit (reverts on failure)
      if (preSolveSubmitData.length > 0) {
```

Gauntlet: Fixed in PR 580.

Cantina Managed: PR 580 adds an explicit `require(!PRICE_FEE_CALCULATOR.isVaultPaused(MULTI_DEPOSITOR_VAULT), Aera__PriceAndFeeCalculatorVaultPaused())` check at the top of `solveRequestsVault()`. An alternative approach of requiring `requests.length != 0` was considered but rejected, as certain branches within `_solveRequestsVault` (e.g., when the request hash is invalid) do not always reach a PFC call that would otherwise revert on a paused vault.